Google Skills

Introduction: From Default to Optimized Configuration

In previous modules, you used default model settings:

According to ADK documentation,
use Pydantic BaseModel to define the exact structure you need:

```python
# From Modules 1-2
agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="..."
)
```

This works for human readers, but causes problems for systems:

- No control over creativity vs consistency
- No safety thresholds configured
- No token limits set
- Same settings for all tasks (creative writing vs. data extraction)

In this module, we'll learn how to configure models for specific requirements using ADK's generate_content_config.

The problem: One size doesn't fit all

Using default settings or the wrong model wastes resources or sacrifices quality:

```
# Problem 1: Expensive model for simple tasks
greeting_agent = LlmAgent(
    model="gemini-2.5-pro",  # Overkill! Costs 10x more than Flash
    instruction="Greet the user warmly"
)

# Problem 2: Wrong temperature for task
factual_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Extract exact dates from documents"
    # No temperature configured - defaults to 1.0 (high creativity/randomness)
    # For factual tasks, we want temperature near 0!
)

# Problem 3: No custom safety configuration
public_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Answer customer questions"
    # Uses default safety settings - may not meet specific compliance needs
)
```

The root problem

Without strategic configuration, agents either waste money, produce inconsistent results, or fail to meet specific safety and compliance requirements.